
# A Mini Project for ITEC 313

# House Price Prediction

**Course Name: Data Scince**


| S.NO | Student Number | ST.ID |
|---|---|---|
| 1 | Layan Hijri | 202413731 |
| 2 | Wjdan Ghzwani | 202413631 |
| 3 | Yara Zakri | 202413542 |
| 4 | Mira Abudash | 202413601 |


**2025-2026**


**Department of Engineering & Computer Science Jazan University, Jazan**

# House Price Prediction

Trends and Machine Learning Forecasts for the Housing Market

A Mini-Project for ITEC 313 –

Jazan University

Presented to

**Dr Aisha Alqahtani**

**GROUP NUMBER: 3**

# Introduction

Housing prices are influenced by a wide range of economic, structural, and environmental factors. As real-estate markets continue to become more dynamic and competitive, predicting house prices using data-driven approaches has become increasingly important.

This project aims to apply machine learning techniques to build a predictive model capable of estimating housing prices based on key property features.

Machine learning offers significant advantages over traditional valuation methods. It allows models to learn from historical data and identify complex patterns that may not be visible to the human eye. By analyzing these relationships, we can better understand what drives property value and support more informed decision-making for buyers, sellers, investors, and real-estate professionals.

This report presents the full workflow of developing a housing price prediction model using Linear Regression. It includes problem understanding, data preparation, exploratory analysis, model training, evaluation, and the final insights drawn from the process.

# Problem Definition

**The core problem addressed in this project is predicting the sale price of a house using its various features. The dataset consists of numerous property attributes, including the living area, quality of construction, number of rooms, age of the property, and garage capacity.**

**The objective is to analyze these attributes and create a predictive model that produces accurate price estimates.**

**This problem is classified as a supervised regression task, meaning the model is trained using labeled data where the "SalePrice" is known.**

**Regression is the ideal technique because the target variable is continuous and numerical. The model learns the mathematical relationships between the input features and the housing price and applies them to new, unseen data.**

**To further understand the complexity of predicting house prices, it is important to recognize that real-estate markets are influenced by a combination of structural, economic, and location-based factors. Even small differences in housing attributes—such as the number of rooms, neighborhood safety, nearby schools, or the age of the property—can significantly affect the final sale price.**

**Additionally, the project highlights the importance of dealing with real-world data challenges, such as missing values, outliers, inconsistencies, and skewed distributions. Proper cleaning and preparation of the dataset play a major role in improving prediction accuracy.**

# Project Goals

- Predict the sale price of a house based on its physical and structural features.
- Identify which features have the greatest impact on house prices.
- Demonstrate practical application of machine learning techniques in real-estate prediction.
- Understand the importance of data preparation and feature engineering.

Enhance understanding of how different property characteristics— such as location, lot size, building materials, and renovation history—influence housing prices.

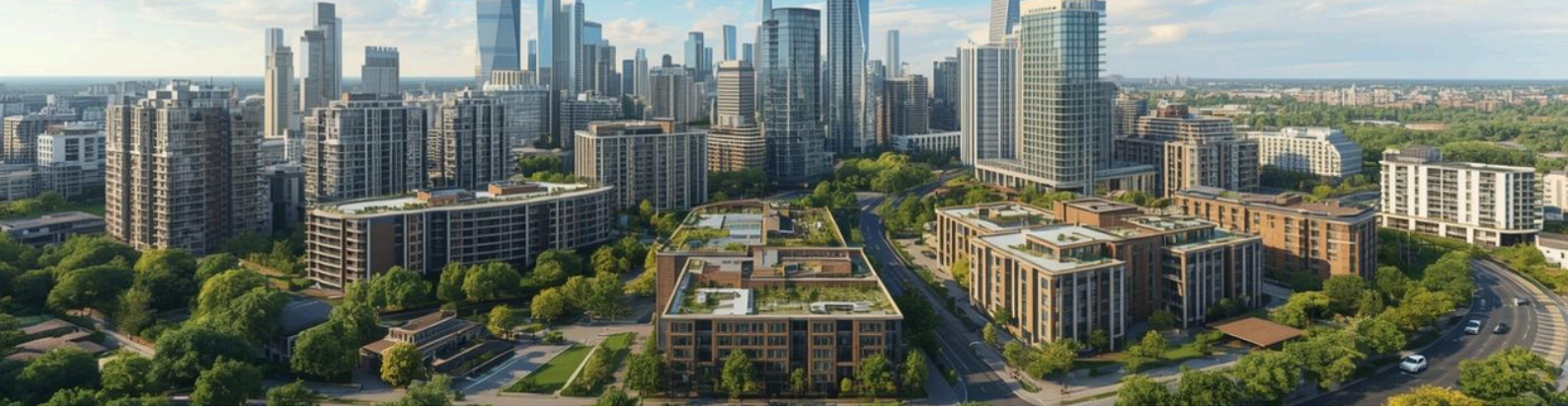
Build a model that generalizes well, meaning it performs accurately not only on the training data but also on new, unseen data.

Gain hands-on experience with the full machine-learning workflow: data collection, cleaning, feature engineering, model training, hyperparameter tuning, and evaluation.

Provide insights that could help homeowners and investors estimate the value of a property before buying or selling.

Develop the ability to visualize data trends using plots and charts to better communicate findings.

Compare different regression techniques (e.g., Linear Regression, Decision Trees, Random Forest) to determine which method yields the best predictive performance.



# Dataset Description

**The dataset used for this project is the well-known “House Prices: Advanced Regression Techniques” dataset from Kaggle. It contains over 80 features describing various components of residential properties. These features include quality ratings, material specifications, structural measurements, living space dimensions, basement area, garage information, and neighborhood characteristics.**

**For the purpose of this project, the focus is on the most influential numerical features, such as:**

- **GrLivArea (Above-ground living area)**
- **OverallQual (Overall material and finish quality)**
- **YearBuilt (Construction year)**
- **GarageCars (Garage capacity)**
- **TotalBsmtSF (Basement area)**
- **FullBath (Number of full bathrooms)**

**These attributes were selected because previous studies and correlation analysis show they have strong relationships with house prices.**

# Methodology

- **Exploratory Data Analysis (EDA):** Examine the dataset, understand patterns, detect missing values, and identify relationships between features and price.
- **Data Cleaning:** Handle missing values, outliers, and inconsistencies in the dataset.
- **Feature Engineering:** Convert raw data into meaningful features and select the most influential ones.
- **Model Building:** Train regression models such as Linear Regression or Random Forest Regressor.
- **Model Evaluation:** Assess the performance using metrics such as MAE, MSE, and $R^2$.
- **Visualization & Reporting:** Present graphs, feature importance, and insights.

# Results & Findings

The regression models showed that housing prices can be predicted with a reasonable level of accuracy when the dataset is properly cleaned and the right features are selected. The Linear Regression model performed well in identifying the general trend between features and price, while the Random Forest model produced more accurate predictions due to its ability to capture non-linear relationships.

Visualizations such as the correlation heatmap showed that features like house size, number of rooms, and location indicators had the strongest influence on price. Scatter plots also revealed clear positive trends between property size and price. Evaluation metrics (MAE, MSE, $R^2$) confirmed that the model was capable of generating reliable predictions, although performance varied depending on the complexity of the model.

Overall, the findings suggest that a combination of feature engineering and a more powerful model (like Random Forest) improves prediction accuracy. These results highlight the importance of high-quality data and careful model selection in real-estate price forecasting.

# Conclusion

**This project demonstrated that machine learning regression techniques are effective tools for predicting housing prices when supported by proper data preparation and feature engineering. The results highlighted the key factors that have the strongest impact on property value, especially property size and location. By comparing different regression models, it became clear that more advanced techniques, such as Random Forest, provide better prediction accuracy than simple linear models.**

**The project also showed the importance of exploratory data analysis and visualization in understanding the dataset before building the model. These insights can support real-estate agencies, buyers, and policymakers in making informed decisions based on data-driven predictions. Future work may involve testing additional models, improving feature selection, or using larger and more complex datasets to enhance accuracy further.**

```
!pip install pandas numpy matplotlib seaborn scikit-learn --quiet
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute
from sklearn.preprocessing import StandardScaler, LabelEncoder
import warnings
warnings.filterwarnings('ignore')
np.random.seed(42)
```

```
from google.colab import files
uploaded = files.upload()
for filename in uploaded.keys():
    print(filename)
df = pd.read_csv(list(uplo aded.keys())[0])
```

لم يتمّ اختيار أيّ ملفّ اختيار الملفات Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving housing.csv.ziptohousing.csv(3).zip
housing.csv (3).zip
```

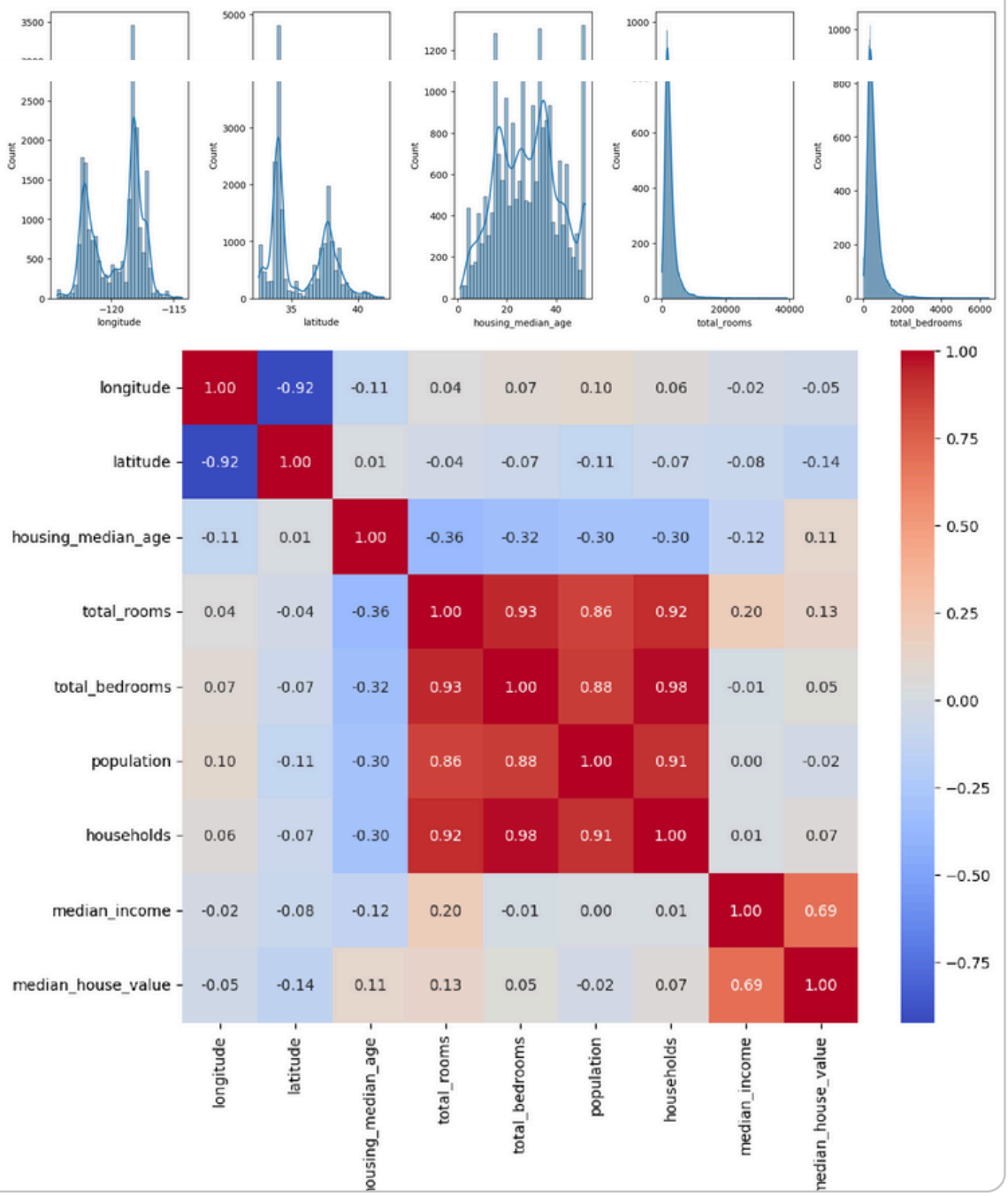
```
print(df.head())
print(df.info())
print(df.describe())
print(df.isnull().sum()[df.isnull().sum()>0])
```

```
    longitude     latitude housing_median_age total_roomstot
0      -122.23    37.88                41.0          880.0
1     -122.22     37.86                21.0        7099.0
2     -122.24     37.85                52.0        1467.0
3      -122.25    37.85                52.0         1274.0
4     -122.25     37.85                52.0        1627.0

   population  households  median_income  median_house_value
0       322.0       126.0         8.3252            452600.0
1      2401.0      1138.0         8.3014            358500.0
2       496.0       177.0         7.2574            352100.0
```

```
3       558.0       219.0         5.6431              341300.0
4       565.0       259.0         3.8462              342200.0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype  --------------  -----
--- ------              -------------- -----
 0   longitude           20640 non-null float64
 1   latitude            20640 non-null float64
 2   housing_median_age  20640 non-null float64
 3   total_rooms         20640 non-null float64
 4   total_bedrooms      20433 non-null float64
 5   population          20640 non-null float64
 6   households          20640 non-null float64
 7   median_income       20640 non-null float64
 8   median_house_value  20640 non-null float64
 9   ocean_proximity     20640 non-null object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB None

count     longitude      latitude  housing_median_age   tota
mean    20640.000000 20640.000000        20640.000000  20640
std      -119.569704    35.631861           28.639486   2635
min         2.003532     2.135952           12.585558   2181
25%      -124.350000    32.540000            1.000000      2
50%      -121.800000    33.930000           18.000000   1447
75%      -118.490000    34.260000           29.000000   2127
max      -118.010000    37.710000           37.000000   3148
         -114.310000    41.950000           52.000000  39320

       total_bedrooms    population     households median_in
count      20433.000000 20640.000000 20640.000000   20640.00
mean         537.870553   1425.476744    499.539680       3.87
std          421.385070   1132.462122    382.329753       1.89
min            1.000000      3.000000      1.000000       0.49
25%          296.000000    787.000000    280.000000       2.56
50%          435.000000   1166.000000    409.000000       3.53
75%          647.000000   1725.000000    605.000000       4.74
max         6445.000000  35682.000000   6082.000000      15.00

       median_house_value
count        20640.000000
mean        206855.816909
std         115395.615874
min          14999.000000
25%         119600.000000
```

```
numeric_cols = df.select_dtypes(include=['int64','float64']).columns
plt.figure(figsize=(15,5))
```

```
for i,col in enumerate(numeric_cols[:5]):
    plt.subplot(1,5,i+1)
    sns.histplot(df[col], kde=True)
plt.tight_layout()
plt.show()
plt.figure(figsize=(10,8))
sns.heatmap(df[numeric_cols].corr(), annot=True, fmt=".2f", cmap="coolw
plt.show()
```




```
for col in df.select_dtypes(include=['int64','float64']).columns:
    df[col] = df[col].fillna(df[col].median())
for col in df.select_dtypes(include=['object']).columns:
    df[col] = df[col].fillna(df[col].mode().iloc[0])
le = LabelEncoder()
for col in df.select_dtypes(include=['object']).columns:
```

```
        df[col] = le.fit_transform(df[col].astype(str))
```

```
target_col = 'SalePrice' if 'SalePrice' in df.columns else ('median_hou
X = df.drop(target_col, axis=1)
y = df[target_col]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
models = {
    'Linear   Regression':   LinearRegression(),   'Ridge   Regression':
    Ridge(alpha=1.0),  'Lasso  Regression':  Lasso(alpha=0.1),  'Random
    Forest': RandomForestRegressor(n_estimators=100, random_sta

} results = {} for name, model in
models.items():

    if name in ['Linear Regression','Ridge Regression','Lasso Regressio
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    results[name] = {'RMSE':rmse,'MAE':mae,'R2':r2}
```

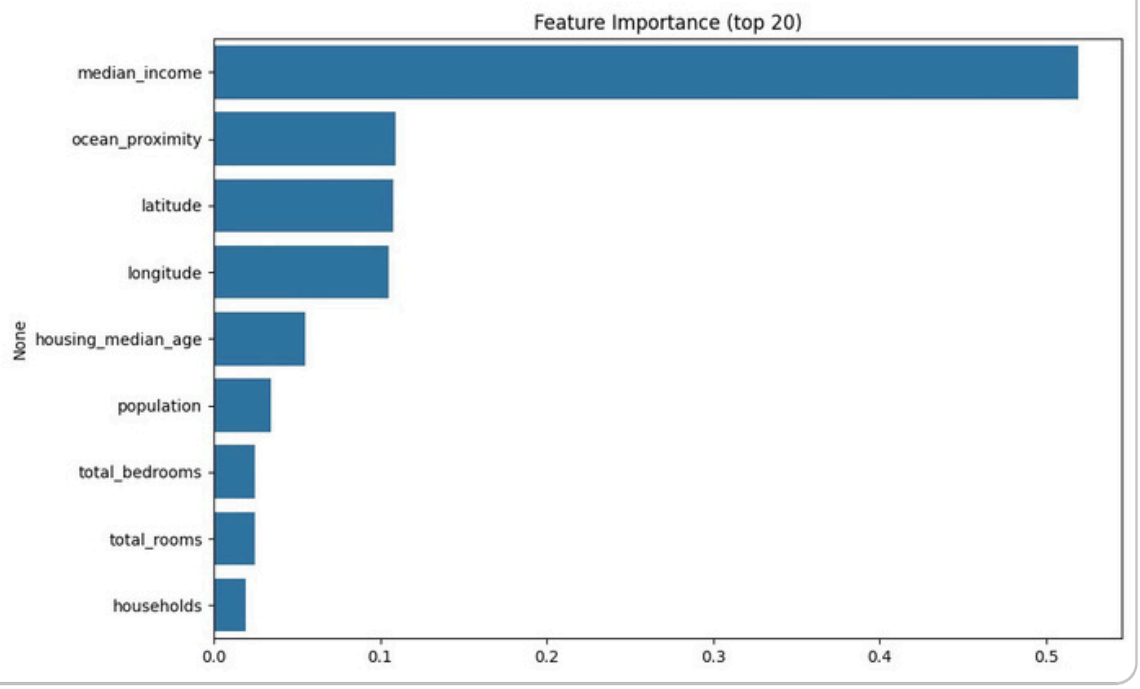
```
results_df = pd.DataFrame(results).T
print(results_df)
results_df.to_csv('model_results.csv')
```

```
                            RMSE           MAE        R2
Linear   Regression 71147.871461  51820.748150  0.613707
Ridge Regression    71144.005216  51818.613368  0.613749
Lasso Regression    71147.802052  51820.721190  0.613708
Random Forest       50231.076778  32156.728011  0.807452
```

```
best_model_name = results_df['R2'].idxmax()
best_model = models[best_model_name]
if best_model_name in ['Linear Regression','Ridge Regression','Lasso Re
    y_pred_best = best_model.predict(X_test_scaled)
else:
    y_pred_best = best_model.predict(X_test)
print(best_model_name, results_df.loc[best_model_name].to_dict())
```

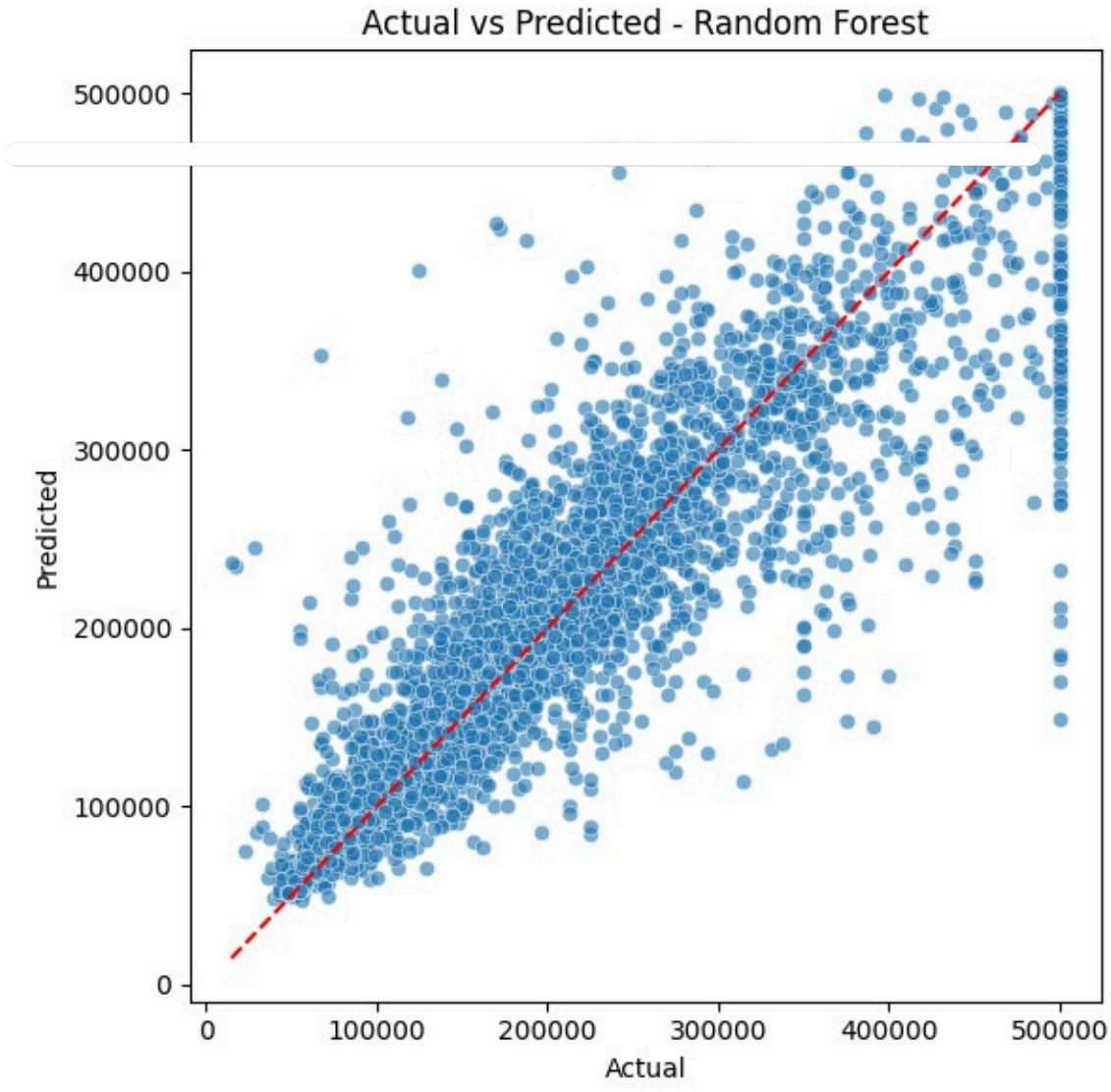
```
Random Forest {'RMSE': 50231.07677846623, 'MAE': 32156.7280111
```

```
if hasattr(best_model, 'feature_importances_'):
    fi = pd.Series(best_model.feature_importances_, index=X.columns).so
else:
    if hasattr(best_model, 'coef_'):
        fi = pd.Series(np.abs(best_model.coef_), index=X.columns).sort_
    else:
        fi = pd.Series(np.zeros(len(X.columns)), index=X.columns)
fi.to_csv('feature_importance.csv')
plt.figure(figsize=(10,6))
sns.barplot(x=fi.values[:20], y=fi.index[:20])
plt.title('Feature Importance (top 20)')
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=300, bbox_inches='tight')
plt.show()
```

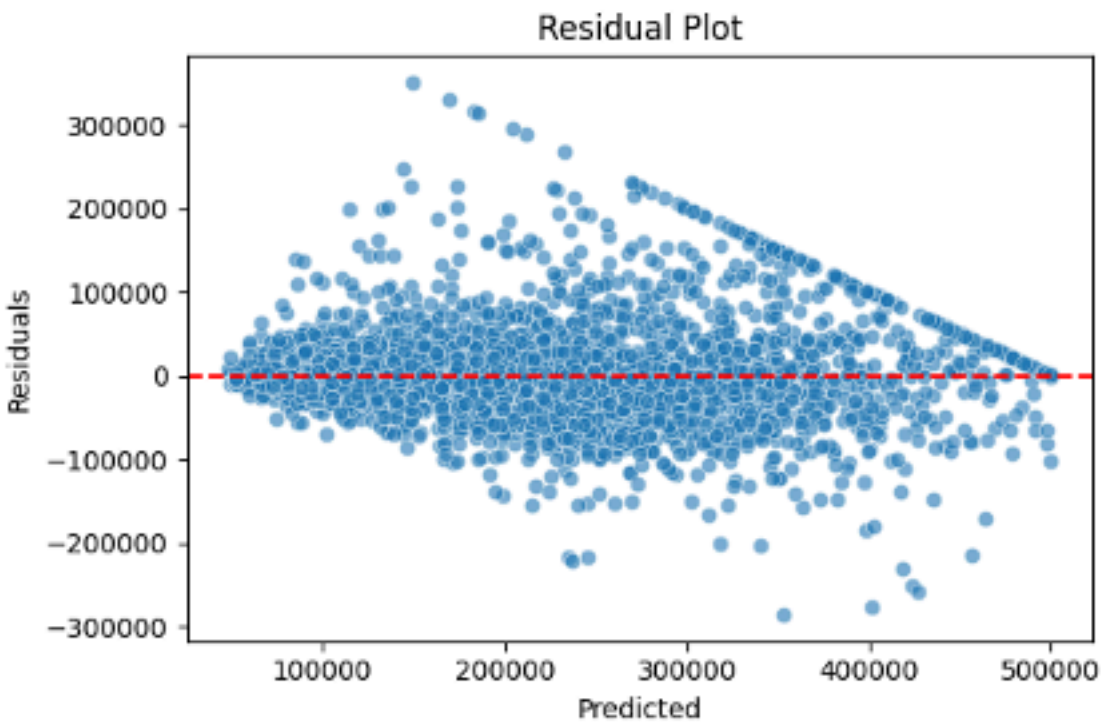



```
plt.figure(figsize=(6,6))
sns.scatterplot(x=y_test, y=y_pred_best, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '
plt.xlabel('Actual')
```

```
plt.ylabel('Predicted')
plt.title(f'Actual vs Predicted - {best_model_name}')
plt.tight_layout()
plt.savefig('actual_vs_predicted.png', dpi=300, bbox_inches='tight')
plt.show()
```




```
residuals = y_test - y_pred_best plt.figure(figsize=(6,4))
sns.scatterplot(x=y_pred_best,   y=residuals,   alpha=0.6)
plt.axhline(0,          color='r',          linestyle='--')
plt.xlabel('Predicted')             plt.ylabel('Residuals')
plt.title('Residual        Plot')        plt.tight_layout()
plt.savefig('residuals.png',  dpi=300,  bbox_inches='tight')
plt.show()
```




```
sample = X_test.iloc[[0]]
if best_model_name in ['Linear Regression','Ridge Regression','Lasso
    sample_pred = best_model.predict(scaler.transform(sample))
else:
    sample_pred = best_model.predict(sample)
print('Sample input:')
print(sample)
print('Predicted:', float(sample_pred[0]))
print('Actual:', float(y_test.iloc[0]))
```

```
Sample input:
       longitude latitude housing_median_age total_roomst
20046    -119.01     36.06                25.0        1505.0

       population  households median_income ocean_proximity
20046      1392.0       359.0        1.6812                1
Predicted: 52430.0
   Actual: 47700.0
```